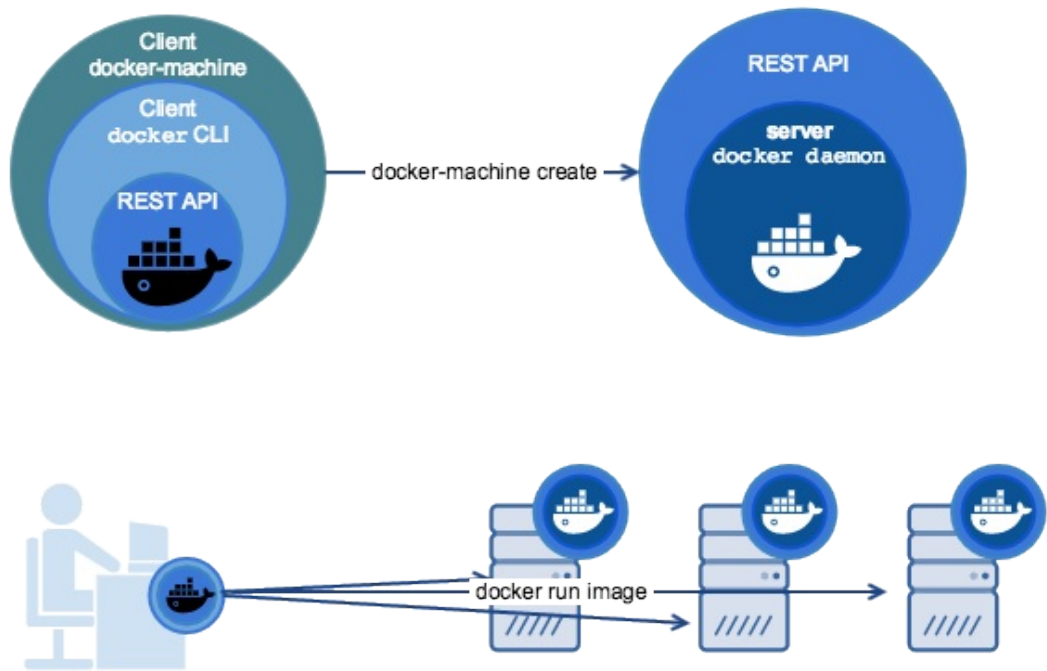


Docker Machine 项目



Docker Machine 是 Docker 官方编排（Orchestration）项目之一，负责在多种平台上快速安装 Docker 环境。

Docker Machine 项目基于 Go 语言实现，目前在 [Github](#) 上进行维护。

本章将介绍 Docker Machine 的安装及使用。

安装

Docker Machine 可以在多种操作系统平台上安装，包括 Linux、macOS，以及 Windows。

macOS、Windows

Docker for Mac、Docker for Windows 自带 `docker-machine` 二进制包，安装之后即可使用。

查看版本信息。

```
$ docker-machine -v
docker-machine version 0.13.0, build 9ba6da9
```

Linux

在 Linux 上的也安装十分简单，从 [官方 GitHub Release](#) 处直接下载编译好的二进制文件即可。

例如，在 Linux 64 位系统上直接下载对应的二进制包。

```
$ sudo curl -L https://github.com/docker/machine/releases/download/v0.13.0/docker-machine-`uname -s`-`uname -m` > /usr/local/bin/docker-machine
$ sudo chmod +x /usr/local/bin/docker-machine
```

完成后，查看版本信息。

```
$ docker-machine -v
docker-machine version 0.13.0, build 9ba6da9
```

使用

Docker Machine 支持多种后端驱动，包括虚拟机、本地主机和云平台等。

创建本地主机实例

Virtualbox 驱动

使用 `virtualbox` 类型的驱动，创建一台 Docker 主机，命名为 `test`。

```
$ docker-machine create -d virtualbox test
```

你也可以在创建时加上如下参数，来配置主机或者主机上的 Docker。

```
--engine-opt dns=114.114.114.114 配置 Docker 的默认 DNS
```

```
--engine-registry-mirror https://registry.docker-cn.com 配置 Docker 的仓库镜像
```

```
--virtualbox-memory 2048 配置主机内存
```

```
--virtualbox-cpu-count 2 配置主机 CPU
```

更多参数请使用 `docker-machine create --driver virtualbox --help` 命令查看。

macOS xhyve 驱动

xhyve 驱动 GitHub: <https://github.com/zchee/docker-machine-driver-xhyve>

`xhyve` 是 macOS 上轻量化的虚拟引擎，使用其创建的 Docker Machine 较 `VirtualBox` 驱动创建的运行效率要高。

```
$ brew install docker-machine-driver-xhyve

$ docker-machine create \
  -d xhyve \
  # --xhyve-boot2docker-url ~/.docker/machine/cache/boot2docker.iso \
  --engine-opt dns=114.114.114.114 \
  --engine-registry-mirror https://registry.docker-cn.com \
  --xhyve-memory-size 2048 \
  --xhyve-rawdisk \
  --xhyve-cpu-count 2 \
  xhyve
```

注意：非首次创建时建议加上 `--xhyve-boot2docker-url`

`~/docker/machine/cache/boot2docker.iso` 参数，避免每次创建时都从 GitHub 下载 ISO 镜像。

更多参数请使用 `docker-machine create --driver xhyve --help` 命令查看。

Windows 10

Windows 10 安装 Docker for Windows 之后不能再安装 VirtualBox，也就不能使用 `virtualbox` 驱动来创建 Docker Machine，我们可以选择使用 `hyperv` 驱动。

```
$ docker-machine create --driver hyperv vm
```

更多参数请使用 `docker-machine create --driver hyperv --help` 命令查看。

使用介绍

创建好主机之后，查看主机

```
$ docker-machine ls
```

NAME	ACTIVE	DRIVER	STATE	URL	SWARM	DOCKER
ERRORS						
test	-	virtualbox	Running	tcp://192.168.99.187:2376		v17.10.0
-ce						

创建主机成功后，可以通过 `env` 命令来让后续操作对象都是目标主机。

```
$ docker-machine env test
```

后续根据提示在命令行输入命令之后就可以操作 `test` 主机。

也可以通过 `SSH` 登录到主机。

```
$ docker-machine ssh test
```

```
docker@test:~$ docker --version
Docker version 17.10.0-ce, build f4ffd25
```

连接到主机之后你就可以在其上使用 Docker 了。

官方支持驱动

通过 `-d` 选项可以选择支持的驱动类型。

- `amazonec2`
- `azure`
- `digitalocean`
- `exoscale`
- `generic`
- `google`
- `hyperv`
- `none`
- `openstack`
- `rackspace`
- `softlayer`
- `virtualbox`
- `vmwarevcloudair`
- `vmwarefusion`
- `vmwarevsphere`

第三方驱动

请到 [第三方驱动列表](#) 查看

操作命令

- `active` 查看活跃的 Docker 主机
- `config` 输出连接的配置信息
- `create` 创建一个 Docker 主机
- `env` 显示连接到某个主机需要的环境变量
- `inspect` 输出主机更多信息
- `ip` 获取主机地址
- `kill` 停止某个主机
- `ls` 列出所有管理的主机
- `provision` 重新设置一个已存在的主机
- `regenerate-certs` 为某个主机重新生成 TLS 认证信息
- `restart` 重启主机
- `rm` 删除某台主机
- `ssh` SSH 到主机上执行命令
- `scp` 在主机之间复制文件
- `mount` 挂载主机目录到本地
- `start` 启动一个主机
- `status` 查看主机状态

- `stop` 停止一个主机
- `upgrade` 更新主机 Docker 版本为最新
- `url` 获取主机的 URL
- `version` 输出 docker-machine 版本信息
- `help` 输出帮助信息

每个命令，又带有不同的参数，可以通过

```
$ docker-machine COMMAND --help
```

来查看具体的用法。

Docker 三剑客之 Docker Swarm

Docker Swarm 是 Docker 官方三剑客项目之一，提供 Docker 容器集群服务，是 Docker 官方对容器云生态进行支持的核心方案。

使用它，用户可以将多个 Docker 主机封装为单个大型的虚拟 Docker 主机，快速打造一套容器云平台。

注意：Docker 1.12.0+ [Swarm mode](#) 已经内嵌入 Docker 引擎，成为了 docker 子命令 `docker swarm`，绝大多数用户已经开始使用 `Swarm mode`，Docker 引擎 API 已经删除 Docker Swarm。为避免大家混淆旧的 `Docker Swarm` 与新的 `Swarm mode`，旧的 `Docker Swarm` 内容已经删除，请查看 `Swarm mode` 一节。

Swarm mode

Docker 1.12 [Swarm mode](#) 已经内嵌入 Docker 引擎，成为了 docker 子命令 `docker swarm`。请注意与旧的 `Docker Swarm` 区分开来。

`Swarm mode` 内置 kv 存储功能，提供了众多的新特性，比如：具有容错能力的去中心化设计、内置服务发现、负载均衡、路由网格、动态伸缩、滚动更新、安全传输等。使得 Docker 原生的 `Swarm` 集群具备与 Mesos、Kubernetes 竞争的实力。

基本概念

`Swarm` 是使用 `SwarmKit` 构建的 `Docker` 引擎内置（原生）的集群管理和编排工具。

使用 `Swarm` 集群之前需要了解以下几个概念。

节点

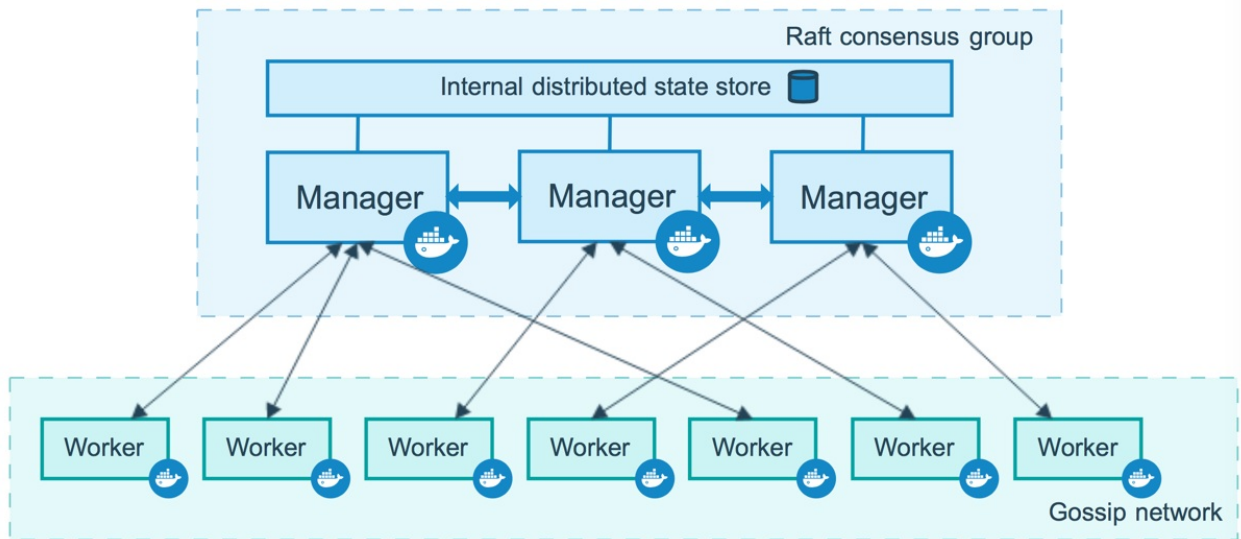
运行 `Docker` 的主机可以主动初始化一个 `Swarm` 集群或者加入一个已存在的 `Swarm` 集群，这样这个运行 `Docker` 的主机就成为一个 `Swarm` 集群的节点（`node`）。

节点分为管理（`manager`）节点和工作（`worker`）节点。

管理节点用于 `Swarm` 集群的管理，`docker swarm` 命令基本只能在管理节点执行（节点退出集群命令 `docker swarm leave` 可以在工作节点执行）。一个 `Swarm` 集群可以有多个管理节点，但只有一个管理节点可以成为 `leader`，`leader` 通过 `raft` 协议实现。

工作节点是任务执行节点，管理节点将服务（`service`）下发至工作节点执行。管理节点默认也作为工作节点。你也可以通过配置让服务只运行在管理节点。

来自 `Docker` 官网的这张图片形象的展示了集群中管理节点与工作节点的关系。



服务和任务

任务（`Task`）是 `Swarm` 中的最小的调度单位，目前来说就是一个单一的容器。

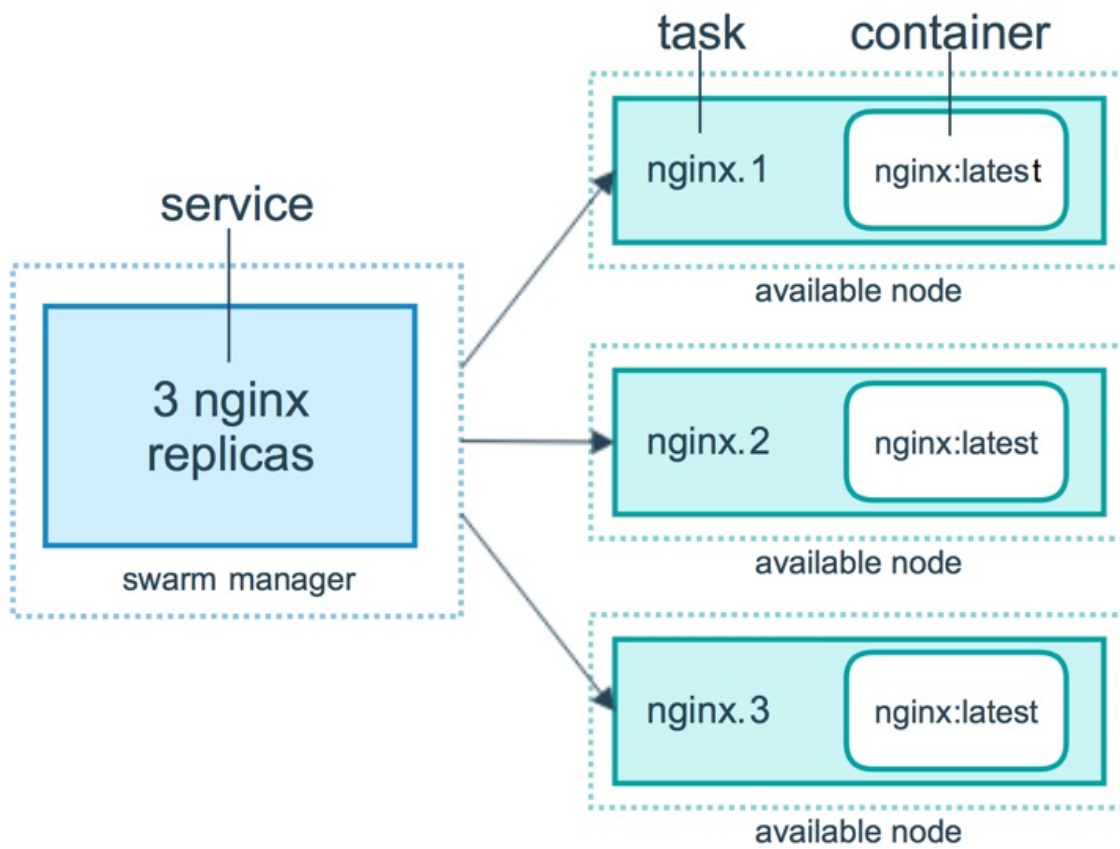
服务（`Services`）是指一组任务的集合，服务定义了任务的属性。服务有两种模式：

- `replicated services` 按照一定规则在各个工作节点上运行指定个数的任务。

- `global services` 每个工作节点上运行一个任务

两种模式通过 `docker service create` 的 `--mode` 参数指定。

来自 Docker 官网的这张图片形象的展示了容器、任务、服务的关系。



创建 Swarm 集群

阅读 [基本概念](#) 一节我们知道 Swarm 集群由管理节点和工作节点组成。本节我们来创建一个包含一个管理节点和两个工作节点的最小 Swarm 集群。

初始化集群

我们使用 `docker swarm init` 在本机初始化一个 Swarm 集群。

```
$ docker swarm init --advertise-addr 192.168.99.100
Swarm initialized: current node (dxn1zf6l61qsb1josjja83ngz) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join \
      --token SWMTKN-1-49nj1cmql0jkz5s954yi3oex3nedyz0fb0xx14ie39trti4wxv-8vxv8rssmk743o
      jnwacrr2e7c \
      192.168.99.100:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the i
nstructions.
```

如果你的 Docker 主机有多个网卡，拥有多个 IP，必须使用 `--advertise-addr` 指定 IP。

执行 `docker swarm init` 命令的节点自动成为管理节点。

增加工作节点

上一步我们在本机初始化了一个 Swarm 集群，拥有了一个管理节点，在 [Docker Machine](#) 一节中我们了解到 Docker Machine 可以在数秒内创建一个虚拟的 Docker 主机，下面我们使用它来创建两个 Docker 主机，并加入到集群中。

```
$ docker-machine create -d virtualbox worker1

$ docker-machine ssh worker1

docker@worker1:~$ docker swarm join \
  --token SWMTKN-1-49nj1cmql0jkz5s954yi3oex3nedyz0fb0xx14ie39trti4wxv-8vxv8rssmk743o
  jnwacrr2e7c \
  192.168.99.100:2377

This node joined a swarm as a worker.
```

```
$ docker-machine create -d virtualbox worker2

$ docker-machine ssh worker2

docker@worker1:~$ docker swarm join \
    --token SWMTKN-1-49nj1cmql0jkz5s954yi3oex3nedyz0fb0xx14ie39trti4wxv-8v xv8rssmk743o
jnwacrr2e7c \
    192.168.99.100:2377

This node joined a swarm as a worker.
```

注意：一些细心的读者可能通过 `docker-machine create --help` 查看到 `--swarm*` 等一系列参数。该参数是用于旧的 `Docker Swarm`，与本章所讲的 `Swarm mode` 没有关系。

查看集群

经过上边的两步，我们已经拥有了一个最小的 `Swarm` 集群，包含一个管理节点和两个工作节点。

在管理节点使用 `docker node ls` 查看集群。

```
$ docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS
03g1y59jwfg7cf99w4lt0f662	worker2	Ready	Active	
9j68exjopxe7wf16yuxml7a7j	worker1	Ready	Active	
dxn1zf6l61qsb1josjja83ngz *	manager1	Ready	Active	Leader

部署服务

我们使用 `docker service` 命令来管理 Swarm 集群中的服务，该命令只能在管理节点运行。

新建服务

现在我们在上一节创建的 Swarm 集群中运行一个名为 `nginx` 服务。

```
$ docker service create --replicas 3 -p 80:80 --name nginx nginx:1.13.7-alpine
```

现在我们使用浏览器，输入任意节点 IP，即可看到 `nginx` 默认页面。

查看服务

使用 `docker service ls` 来查看当前 Swarm 集群运行的服务。

```
$ docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE
	PORTS			
kc57xffvhul5	nginx	replicated	3/3	nginx:
1.13.7-alpine	*:80->80/tcp			

使用 `docker service ps` 来查看某个服务的详情。

```
$ docker service ps nginx
```

ID	NAME	IMAGE	NODE	DESI
RED STATE	CURRENT STATE	ERROR	PORTS	
pjfdz39buzlt	nginx.1	nginx:1.13.7-alpine	swarm2	Runn
ing	Running about a minute ago			
hy9eeivdxlaa	nginx.2	nginx:1.13.7-alpine	swarm1	Runn
ing	Running about a minute ago			
36wmpiv7gmfo	nginx.3	nginx:1.13.7-alpine	swarm3	Runn
ing	Running about a minute ago			

使用 `docker service logs` 来查看某个服务的日志。

```
$ docker service logs nginx
nginx.3.36wmpiv7gmfo@swarm3 | 10.255.0.4 - - [25/Nov/2017:02:10:30 +0000] "GET / HTTP/1.1" 200 612 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.13; rv:58.0) Gecko/20100101 Firefox/58.0" "-"
nginx.3.36wmpiv7gmfo@swarm3 | 10.255.0.4 - - [25/Nov/2017:02:10:30 +0000] "GET /favicon.ico HTTP/1.1" 404 169 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.13; rv:58.0) Gecko/20100101 Firefox/58.0" "-"
nginx.3.36wmpiv7gmfo@swarm3 | 2017/11/25 02:10:30 [error] 5#5: *1 open() "/usr/share/nginx/html/favicon.ico" failed (2: No such file or directory), client: 10.255.0.4, server: localhost, request: "GET /favicon.ico HTTP/1.1", host: "192.168.99.102"
nginx.1.pjfzd39buzlt@swarm2 | 10.255.0.2 - - [25/Nov/2017:02:10:26 +0000] "GET / HTTP/1.1" 200 612 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.13; rv:58.0) Gecko/20100101 Firefox/58.0" "-"
nginx.1.pjfzd39buzlt@swarm2 | 10.255.0.2 - - [25/Nov/2017:02:10:27 +0000] "GET /favicon.ico HTTP/1.1" 404 169 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.13; rv:58.0) Gecko/20100101 Firefox/58.0" "-"
nginx.1.pjfzd39buzlt@swarm2 | 2017/11/25 02:10:27 [error] 5#5: *1 open() "/usr/share/nginx/html/favicon.ico" failed (2: No such file or directory), client: 10.255.0.2, server: localhost, request: "GET /favicon.ico HTTP/1.1", host: "192.168.99.101"
```

删除服务

使用 `docker service rm` 来从 `Swarm` 集群移除某个服务。

```
$ docker service rm nginx
```

在 Swarm 集群中使用 compose 文件

正如之前使用 `docker-compose.yml` 来一次配置、启动多个容器，在 Swarm 集群中也可以使用 `compose` 文件（`docker-compose.yml`）来配置、启动多个服务。

上一节中，我们使用 `docker service create` 一次只能部署一个服务，使用 `docker-compose.yml` 我们可以一次启动多个关联的服务。

我们以在 Swarm 集群中部署 WordPress 为例进行说明。

```
version: "3"

services:
  wordpress:
    image: wordpress
    ports:
      - 80:80
    networks:
      - overlay
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wordpress
      WORDPRESS_DB_PASSWORD: wordpress
    deploy:
      mode: replicated
      replicas: 3

  db:
    image: mysql
    networks:
      - overlay
    volumes:
      - db-data:/var/lib/mysql
    environment:
      MYSQL_ROOT_PASSWORD: somewordpress
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wordpress
      MYSQL_PASSWORD: wordpress
    deploy:
      placement:
        constraints: [node.role == manager]

  visualizer:
    image: dockersamples/visualizer:stable
    ports:
      - "8080:8080"
    stop_grace_period: 1m30s
    volumes:
      - "/var/run/docker.sock:/var/run/docker.sock"
    deploy:
      placement:
        constraints: [node.role == manager]

volumes:
  db-data:

networks:
  overlay:
```

在 `Swarm` 集群管理节点新建该文件，其中的 `visualizer` 服务提供一个可视化页面，我们可以从浏览器中很直观的查看集群中各个服务的运行节点。

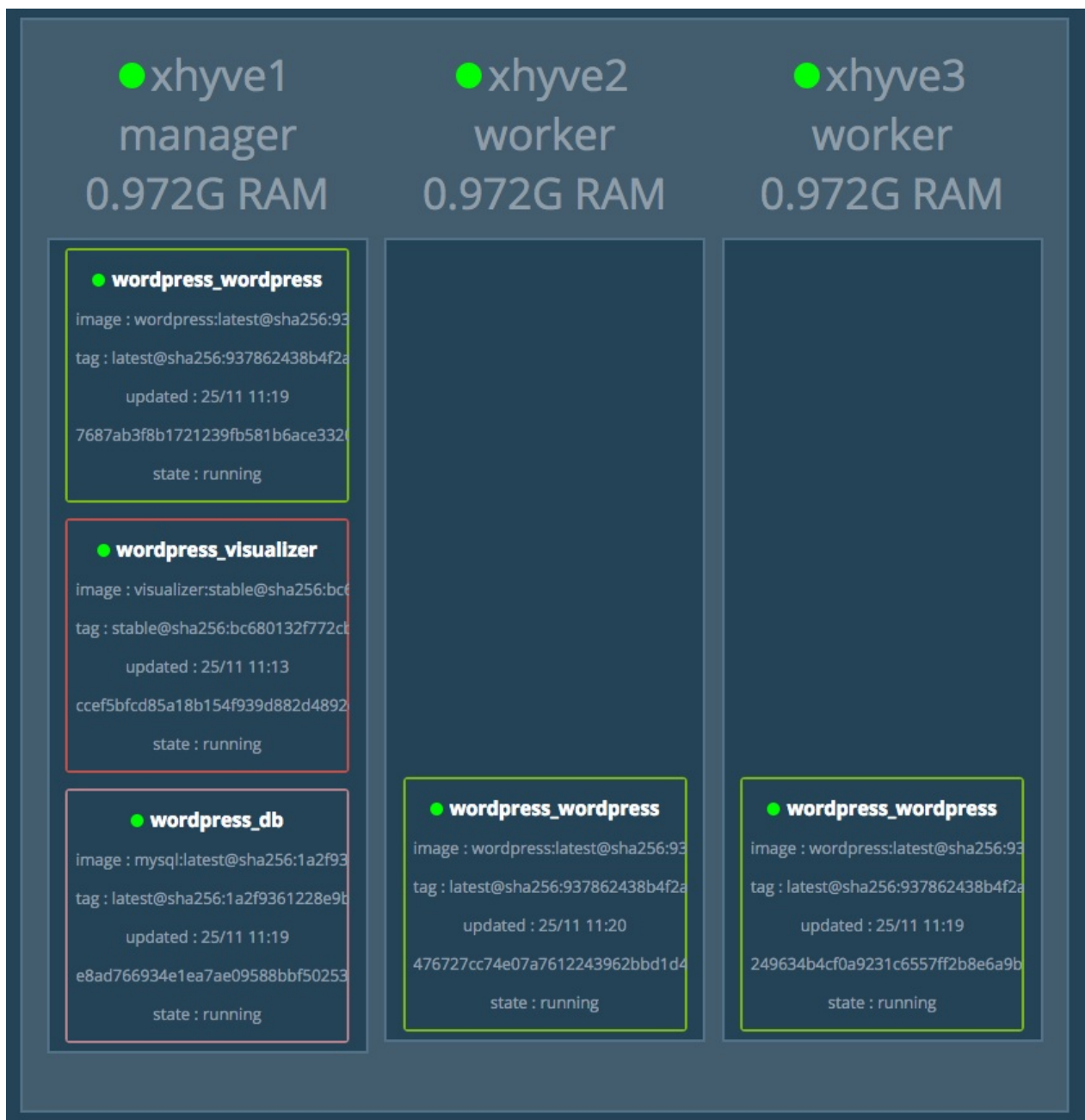
在 Swarm 集群中使用 `docker-compose.yml` 我们用 `docker stack` 命令，下面我们对该命令进行详细讲解。

部署服务

部署服务使用 `docker stack deploy`，其中 `-c` 参数指定 `compose` 文件名。

```
$ docker stack deploy -c docker-compose.yml wordpress
```

现在我们打开浏览器输入 任一节点IP:8080 即可看到各节点运行状态。如下图所示：



在浏览器新的标签页输入 任一节点IP 即可看到 WordPress 安装界面，安装完成之后，输入 任一节点IP 即可看到 WordPress 页面。

查看服务

```
$ docker stack ls
NAME                SERVICES
wordpress           3
```

移除服务

要移除服务，使用 `docker stack down`

```
$ docker stack down wordpress
Removing service wordpress_db
Removing service wordpress_visualizer
Removing service wordpress_wordpress
Removing network wordpress_overlay
Removing network wordpress_default
```

该命令不会移除服务所使用的 `数据卷`，如果你想移除数据卷请使用 `docker volume rm`

在 Swarm 集群中管理敏感数据

在动态的、大规模的分布式集群上，管理和分发 密码 、 证书 等敏感信息是极其重要的工作。传统的密钥分发方式（如密钥放入镜像中，设置环境变量，volume 动态挂载等）都存在着潜在的巨大的安全风险。

Docker 目前已经提供了 `secrets` 管理功能，用户可以在 Swarm 集群中安全地管理密码、密钥证书等敏感数据，并允许在多个 Docker 容器实例之间共享访问指定的敏感数据。

注意： `secret` 也可以在 Docker Compose 中使用。

我们可以用 `docker secret` 命令来管理敏感信息。接下来我们在上面章节中创建好的 Swarm 集群中介绍该命令的使用。

这里我们以在 Swarm 集群中部署 `mysql` 和 `wordpress` 服务为例。

创建 secret

我们使用 `docker secret create` 命令以管道符的形式创建 `secret`

```
$ openssl rand -base64 20 | docker secret create mysql_password -  
$ openssl rand -base64 20 | docker secret create mysql_root_password -
```

查看 secret

使用 `docker secret ls` 命令来查看 `secret`

```
$ docker secret ls
```

ID	NAME	CREATED	UPDATED
l1vinzevzhj4goakjap5ya409	mysql_password	41 seconds ago	41 seconds ago
yvsczlx9votfw3l0nz5rlidig	mysql_root_password	12 seconds ago	12 seconds ago

创建 MySQL 服务

创建服务相关命令已经在前边章节进行了介绍，这里直接列出命令。

```
$ docker network create -d overlay mysql_private

$ docker service create \
  --name mysql \
  --replicas 1 \
  --network mysql_private \
  --mount type=volume,source=mydata,destination=/var/lib/mysql \
  --secret source=mysql_root_password,target=mysql_root_password \
  --secret source=mysql_password,target=mysql_password \
  -e MYSQL_ROOT_PASSWORD_FILE="/run/secrets/mysql_root_password" \
  -e MYSQL_PASSWORD_FILE="/run/secrets/mysql_password" \
  -e MYSQL_USER="wordpress" \
  -e MYSQL_DATABASE="wordpress" \
  mysql:latest
```

如果你没有在 `target` 中显式的指定路径时，`secret` 默认通过 `tmpfs` 文件系统挂载到容器的 `/run/secrets` 目录中。

```
$ docker service create \
  --name wordpress \
  --replicas 1 \
  --network mysql_private \
  --publish target=30000,port=80 \
  --mount type=volume,source=wpdata,destination=/var/www/html \
  --secret source=mysql_password,target=wp_db_password,mode=0400 \
  -e WORDPRESS_DB_USER="wordpress" \
  -e WORDPRESS_DB_PASSWORD_FILE="/run/secrets/wp_db_password" \
  -e WORDPRESS_DB_HOST="mysql:3306" \
  -e WORDPRESS_DB_NAME="wordpress" \
  wordpress:latest
```

查看服务

```
$ docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE
wvnh0siktqr3	mysql	replicated	1/1	mysql:latest
nzt5xzae4n62	wordpress	replicated	1/1	wordpress:latest

现在浏览器访问 `IP:30000`，即可开始 `WordPress` 的安装与使用。

通过以上方法，我们没有像以前通过设置环境变量来设置 `MySQL` 密码，而是采用 `docker secret` 来设置密码，防范了密码泄露的风险。

在 Swarm 集群中管理配置数据

在动态的、大规模的分布式集群上，管理和分发配置文件也是很重要的工作。传统的配置文件分发方式（如配置文件放入镜像中，设置环境变量，`volume` 动态挂载等）都降低了镜像的通用性。

在 Docker 17.06 以上版本中，Docker 新增了 `docker config` 子命令来管理集群中的配置信息，以后你无需将配置文件放入镜像或挂载到容器中就可实现对服务的配置。

注意：`config` 仅能在 Swarm 集群中使用。

这里我们以在 Swarm 集群中部署 `redis` 服务为例。

创建 config

新建 `redis.conf` 文件

```
port 6380
```

此项配置 Redis 监听 `6380` 端口

我们使用 `docker config create` 命令创建 `config`

```
$ docker config create redis.conf redis.conf
```

查看 config

使用 `docker config ls` 命令来查看 `config`

```
$ docker config ls
```

ID	NAME	CREATED	UPDATED
yod8fx8iiqtoo84jgwadp86yk	redis.conf	4 seconds ago	4 seconds ago

创建 redis 服务

```
$ docker service create \  
  --name redis \  
  # --config source=redis.conf,target=/etc/redis.conf \  
  --config redis.conf \  
  -p 6379:6380 \  
  redis:latest \  
  redis-server /redis.conf
```

如果你没有在 `target` 中显式的指定路径时，默认的 `redis.conf` 以 `tmpfs` 文件系统挂载到容器的 `/config.conf`。

经过测试，`redis` 可以正常使用。

以前我们通过监听主机目录来配置 `Redis`，就需要在集群的每个节点放置该文件，如果采用 `docker config` 来管理服务的配置信息，我们只需在集群中的管理节点创建 `config`，当部署服务时，集群会自动的将配置文件分发到运行服务的各个节点中，大大降低了配置信息的管理和分发难度。